



Développer pour le Cloud

2025-26 Valentin PICARD
DEVFLSTK MAST2 B



2015 — Helm naît dans un hackathon

Deux ingénieurs — Matt Butcher et Jack Francis — se rendent compte que déployer des apps complexes sur Kubernetes revient à écrire des dizaines de fichiers YAML à la main.

Ils créent un gestionnaire de packages pour Kubernetes et l'appellent **Helm**

- En 2019, Helm rejoint la **CNCF**.
- En 2020, Helm 3 est publié (version utilisée aujourd'hui).

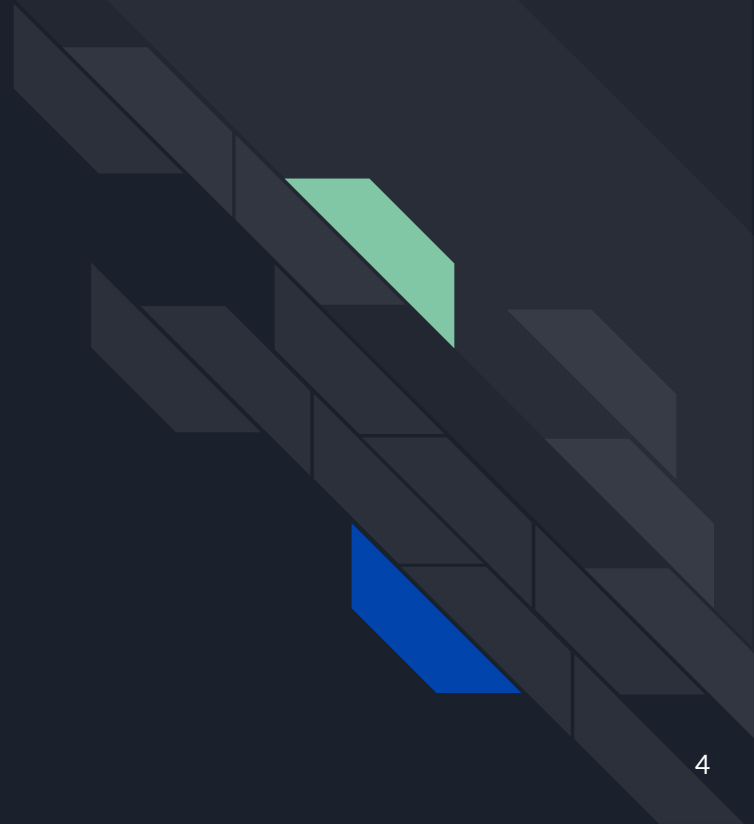


HoneyPot — Kubernetes Documentary (2022)

Série documentaire en deux parties avec les fondateurs :

- [Partie 1 - disponible sur YouTube](#)
- [Partie 2 - disponible sur YouTube](#)

Helm





HELM

1. Helm c'est quoi ?
2. Transition vers Helm
3. Les charts
4. values.yml
5. Dépendances Helm
6. Release
7. helm-template
8. helm-diff
9. Trois niveaux de versionning



1 - Helm c'est quoi ?

Helm, c'est le gestionnaire de packages de Kubernetes

Situation : Tu veux installer Prometheus sur ton cluster.

- C'est complexe, ça représente des dizaines de fichiers YAML.
- La communauté a fait ce travail et l'a empaqueté dans un **Chart**.



1 - Pourquoi HELM ?

Quand Helm devient pertinent ?

1. Multi-environnement sans duplication

- Avec du YAML brut, 3 environnements = 3 copies quasi-identiques à maintenir.
- Avec Helm : une seule source de vérité + 3 fichiers de valeurs.



1 - Pourquoi HELM ?

Quand Helm devient pertinent ?

2. Installation atomique

- `helm install` déploie tout en une commande.
- Avec `kubectl apply -f <fichier.yaml>` il faut sélectionner tous les fichiers
- Avec l'option `--recursive`, l'ordonnancement n'est pas garanti



1 - Pourquoi HELM ?

Quand Helm devient pertinent ?

3. Packaging et partage

- Un chart est un artefact versionné, publiable sur un registry.
- C'est exactement comme ça que l'on peut installer Prometheus ou ingress-nginx en une ligne.

Impact sur le projet
Taskflow ?



2 - Transition vers Helm

Pour déployer TaskFlow complet, il faut :

task-service/ configmap.yaml	deployment.yaml + service.yaml +
user-service/ configmap.yaml	deployment.yaml + service.yaml +
notification/ configmap.yaml	deployment.yaml + service.yaml +
api-gateway/ configmap.yaml	deployment.yaml + service.yaml +
frontend/	deployment.yaml + service.yaml
postgres/	statefulset.yaml + service.yaml
redis/	deployment.yaml + service.yaml
ingress.yaml	



2 - Transition vers Helm

Factoriser les yaml par services en utilisant le séparateur ---

Exemple avec `task-service.yaml`

```
apiVersion: apps/v1
kind: Deployment
[...]
---                                # ← séparateur
apiVersion: v1
kind: Service
[...]
---                                # ← séparateur
apiVersion: v1
kind: ConfigMap
[...]
```



2 - Transition vers Helm

Pour chaque service TaskFlow, il existe autant de yaml que d'environnements:

```
<service>.staging.yaml  
<service>.production.yaml  
<service>.dev.yaml
```

Le vrai problème :

- 80% du YAML est identique entre les services applicatifs
- Changer le tag d'image → modifier 5 fichiers
- Passer en production avec `replicas: 3` → dupliquer tout

2 - Transition vers Helm

```
helm/taskflow/
├── Chart.yaml
├── values.yaml           ← valeurs communes
├── values.staging.yaml  ← surcharges staging (debug, replicas: 1)
├── values.production.yaml ← surcharges production (replicas: 3)
└── templates/
    ├── task-service.yaml ← Deployment + Service avec {{ variables
    }}
    ├── user-service.yaml
    ├── notification-service.yaml
    ├── api-gateway.yaml
    ├── frontend.yaml
    ├── postgres.yaml
    ├── redis.yaml
    └── ingress.yaml
```



```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: task-service
  namespace: {{ .Release.Namespace }}
spec:
  replicas: {{ .Values.taskService.replicaCount }}
  selector:
    matchLabels:
      app: task-service
  template:
    metadata:
      labels:
        app: task-service
    spec:
      containers:
        - name: task-service
          image: "{{ .Values.image.prefix }}-task-service:
            {{ .Values.image.tag }}"
```



```
# values.yaml  
replicaCount: 1  
image:  
  prefix: <user>/taskflow  
  tag: latest
```

```
# values.production.yaml – seulement ce qui change  
replicaCount: 3  
image:  
  tag: v1.2.0
```



2 - Secrets dans les values Helm

Ci-contre, un fichier `values.production.yaml`.

Voyez-vous un problème ?



```
# values.production.yaml
replicaCount: 3
resources:
  limits:
    memory: 512Mi
jwt:
  secret: mon-super-secret-jwt
postgres:
  password: taskflow-prod-password
```



2 - Secrets dans les values Helm

- Créer le fichier de secrets : `values.secret.yaml`

```
postgres:
  password: "monmotdepasse"
jwt:
  secret: "monsecretjwt"
```

- L'ajouter au `.gitignore`



2 - Secrets dans les values Helm

- Fournir un exemple committé : `values.secret.yaml.example`

```
postgres:
  password: ""
jwt:
  secret: ""
```

- Lancer Helm avec les deux fichiers

```
helm upgrade --install taskflow ./helm/taskflow \
  --values ./helm/taskflow/values.yaml \
  --values ./helm/taskflow/values.secret.yaml
```



2 - Secrets dans les values Helm

- Fournir un exemple committé : `values.secret.yaml.example`

```
postgres:
  password: ""
jwt:
  secret: ""
```

- Lancer Helm avec les deux fichiers

```
helm upgrade --install taskflow ./helm/taskflow \
  --values ./helm/taskflow/values.yaml \
  --values ./helm/taskflow/values.secret.yaml
```



2 - Secrets dans les values Helm

La solution : passer les secrets via `--set` au déploiement :

```
- name: Deploy
  run: |
    helm upgrade --install taskflow ./helm/taskflow \
      --values ./helm/taskflow/values.yaml \
      --set postgres.password=${{ secrets.POSTGRES_PASSWORD }} \
      --set jwt.secret=${{ secrets.JWT_SECRET }}
```

`$POSTGRES_PASSWORD` et `$JWT_SECRET` sont stockés dans Settings → Secrets → Actions sur GitHub.



3 - Utiliser des charts officiels

```
helm install postgres bitnami/postgresql ...  
helm install redis bitnami/redis ...  
helm install monitoring prometheus-community/kube-prometheus-stack ...
```



3 - Un Chart, c'est quoi ?

Le **Chart**, c'est un dossier qui contient :

```
kube-prometheus-stack/ ← nom du chart (fixe, défini par ses auteurs)
├─Chart.yaml             ← nom, version, métadonnées du chart
├─values.yaml            ← toutes les options configurables avec leurs
├─                        valeurs par défaut
└─ templates/            ← les fichiers YAML avec des {{ variables }}
    ├─deployment.yaml
    ├─service.yaml
    └─ ...
```

On a un dossier `templates/`,
un `Chart.yaml`, un
`values.yaml`.

Selon vous, que se passe-t-il
concrètement quand vous
faites `helm install`, dans
quel ordre Helm lit ces trois
fichiers ?



4 - values.yaml

Le `values.yaml` du chart est fourni par les auteurs.

Il contient les valeurs par défaut :

```
# values.yaml du chart kube-prometheus-stack
grafana:
  adminPassword: "prom-operator"  # valeur par défaut
  replicas: 1
```



4 - values.yaml

Utilise ton propre fichier pour changer ce qui ne te convient pas.

Ce fichier, comme les secrets, ne doit pas être commité.

```
# my-values.yaml
grafana:
  adminPassword: "mon-mot-de-passe"
```

Helm fusionne les deux, génère tous les YAML et les applique sur le cluster :

```
helm install monitoring prometheus-community/kube-prometheus-stack \
  --values mes-valeurs.yaml
```



5 - Dépendances Helm

Un chart Helm peut déclarer des “**sous-charts**” comme dépendances.

Au lieu d'écrire un template Redis maison, on délègue à un chart Bitnami maintenu par la communauté.



```
#Chart.yaml
```

```
dependencies:
```

```
- name: redis
```

```
  version: "18.x.x"
```

```
  repository: https://charts.bitnami.com/bitnami
```

```
  condition: redis.enabled
```

```
    # permet de désactiver via values si besoin
```

```
---
```

```
helm dependency update ./helm/taskflow
```

```
# → crée le dossier charts/ avec les archives .tgz des dépendances
```

```
---
```

```
# values.yaml
```

```
redis:
```

```
  enabled: true
```

```
  fullnameOverride: redis    # force le nom du Service à "redis"
```

```
  architecture: standalone
```

```
  auth:
```

```
    enabled: false
```



5 - Dépendances Helm

Chart officiel vs template maison, comment choisir ?

La règle :

- Chart officiel → service **stateless** ou **générique** (Redis, un proxy, un cache)
- Template maison → service portant de la **logique métier** (schéma de DB custom, config spécifique)

Un chat officiel :

- est un gain net quand le service est interchangeable
- devient coûteux quand il porte de la connaissance sur votre application.



6 - Release

```
helm install monitoring                # ← nom de la release (tu choisis)
  prometheus-community/kube-prometheus-stack # ← nom du chart (fixe)
  --values mes-valeurs.yaml             # ← tes valeurs
```

Ce que Helm enregistre dans le cluster sous le nom **monitoring** :

- Le chart utilisé et sa version
- Les `values` utilisées
- La liste de toutes les ressources créées



6 - Release

Helm se souvient de ce qu'il a fait.

Sans Helm, `rollout history` est peu lisible sans discipline :

```
kubectl rollout history deployment/task-service -n staging
# REVISION  CHANGE-CAUSE
# 1          <none>          ← inutilisable sans annotation manuelle
```

Avec Helm, l'historique est automatique et structuré :

```
helm history taskflow -n staging
```

#	REVISION	UPDATED	STATUS	CHART	DESCRIPTION
# 1		2024-01-15 10:23:41 UTC	deployed	taskflow-0.1.0	Install complete
# 2		2024-01-15 14:05:12 UTC	deployed	taskflow-0.1.0	Upgrade complete
# 3		2024-01-15 16:30:55 UTC	failed	taskflow-0.1.0	Upgrade failed: ...



6 - Release

Exemple d'une commande de mise à jour :

```
helm upgrade task-service ./chart \  
  --set image.tag=1.1.0 \  
  --description "fix bug login" \  
  -n staging
```

- `task-service` — le nom de la **release** Helm (pas le nom du chart)
- `./chart` — le chemin vers le chart à déployer
- `--set image.tag=1.1.0` — surcharge une valeur du `values.yaml` à la volée, ici la tag de l'image
- `--description "fix bug login"` — message libre qui apparaît dans colonne `DESCRIPTION` après `helm history`



6 - Release

Les commandes de gestion de release :

Helm sait quoi mettre à jour

```
helm upgrade monitoring ...
```

Historique complet et lisible

```
helm history monitoring -n monitoring
```

Revenir à la révision 1

```
helm rollback monitoring 1
```

Helm sait quoi supprimer

```
helm uninstall monitoring
```



7-helm template — voir avant d'appliquer

`helm template` génère le YAML final que Helm enverrait au cluster (sans rien appliquer).

```
helm template taskflow ./helm/taskflow \  
  --namespace staging \  
  --values ./helm/taskflow/values.staging.yaml
```



```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: task-service
  namespace: {{ .Release.Namespace }}
spec:
  replicas: {{ .Values.taskService.replicaCount }}
  selector:
    matchLabels:
      app: task-service
  template:
    metadata:
      labels:
        app: task-service
    spec:
      containers:
        - name: task-service
          image: "{{ .Values.image.prefix }}-task-service:
            {{ .Values.image.tag }}"
```



Résultat – les {{ variables }} ont été remplacées par les vraies valeurs

apiVersion: apps/v1

kind: Deployment

metadata:

name: task-service

namespace: staging

spec:

replicas: 1

← vient de values.staging.yaml

template:

spec:

containers:

- image: monuser/taskflow-task-service:v1.2.0

← vient de values.yaml

resources:

limits:

memory: 256Mi

← vient de values.staging.yaml



7-helm template — voir avant d'appliquer

Pourquoi c'est important :

- Vérifier que les variables sont bien substituées avant de toucher le cluster
- Déboguer un template qui ne compile pas
- Comparer ce que Helm génère avec ce qu'on écrirait à la main

Filtrer sur un seul template :

```
helm template taskflow ./helm/taskflow \  
  --values ./helm/taskflow/values.staging.yaml \  
  --show-only templates/task-service.yaml
```



8-helm diff — voir ce qui va changer

`helm template` montre ce que Helm générerait.

`helm diff` montre la différence entre ce qui tourne dans le cluster et ce qui va être appliqué.

Installer le plugin une fois :

```
helm plugin install https://github.com/databus23/helm-diff
```

Avant chaque upgrade :

```
helm diff upgrade taskflow ./helm/taskflow \  
  --namespace staging \  
  --values ./helm/taskflow/values.staging.yaml
```



8-helm diff — voir ce qui va changer

Ce qu'on voit en sortie comme un `git diff` :

```
deployment.apps/task-service (apps) has changed:
  spec:
    replicas: 1      # ← avant
+   replicas: 2      # ← après
```

Pourquoi c'est critique :

- Changer `replicaCount` est inoffensif.
- Changer `image.tag` ou une variable d'environnement peut casser des Pods en production.
- `helm diff` permet de valider l'impact avant que le cluster soit affecté.



9 - Trois niveaux de versioning

Tag Docker

v1.0.0

v1.1.0

v1.1.0

Révision Kubernetes

REVISION 1

REVISION 2

REVISION 3 ← même
image,
config changée

Révision Helm

REVISION 1

REVISION 2

REVISION 3 ← changement
de config,
pas d'image



9 - Trois niveaux de versioning

Tag Docker (`v1.0.0`), version du code applicatif:

- Défini dans le `Dockerfile` ou poussé par la CI
- Immuable une fois pushé sur Docker Hub
- Rollback : changer le tag dans le Deployment



9 - Trois niveaux de versioning

Révision Kubernetes (`rollout history`), version du Deployment dans le cluster :

- Créée à chaque `kubectl apply` ou `kubectl set image`
- Peut changer sans que l'image change (ex: variable d'env modifiée)
- Rollback: `kubectl rollout undo`



9 - Trois niveaux de versioning

Révision Helm (`helm history`), version de la release Helm

- Créée à chaque `helm upgrade`
- Englobe tout : image, config, nombre de replicas...
- Rollback: `helm rollback taskflow 1`



9 - Trois niveaux de versioning

Ce que ça donne en investigation d'incident :

- Grafana montre une erreur à 14h32
 - `helm history: upgrade` à 14h30 (révision 5)
 - `rollout history` : **CHANGE-CAUSE** "image v1.2.0 - nouvelle feature"
 - `docker pull v1.2.0`: le bug est dans ce tag



Ressources utiles

Helm — officiel

- helm.sh/docs — la référence complète : commandes, templating, dépendances, hooks
- [Artifact Hub](https://artifacthub.io) — le registry public de charts Helm (Bitnami, prometheus-community, ingress-nginx...)
- [helm-diff plugin](#) — voir ce qui va changer avant chaque upgrade

Charts à connaître

- [kube-prometheus-stack](#) — Prometheus + Grafana + Alertmanager en une commande
- [ingress-nginx](#) — le chart officiel pour l'Ingress Controller
- [bitnami/redis](#) et [bitnami/postgresql](#) — les charts de référence pour les services stateful

Pour aller plus loin

- [Chart Testing \(ct\)](#) — outil de lint et test de charts en CI
- [Helmfile](#) — décrire et orchestrer plusieurs releases Helm dans un seul fichier déclaratif